SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

This section introduces our core AI engine: The Data Detective and the Strategic Sandbox. We use AI to solve two massive problems for F&B owners: making sense of messy past data, and the high cost of future trial-and-error.

First, Tauke.AI acts as your data detective, instantly analyzing past receipts, invoices, and market signals to uncover hidden profit leaks you didn't even know existed. But we don't just look backward. We then give owners a digital playground to test the future. Instead of relying on gut feelings, you can run 'What if?' scenarios—like launching a flash sale or raising prices—through our AI customer swarm. It instantly predicts the profit impact and operational risks, turning complex data into a safe, step-by-step execution plan before you spend a single Ringgit in the real world.

1.1. Purpose:

This system analysis document highlights the technical scope and design-related decisions behind the development of Tauke.AI.

So, the key element of system that has been covered are as followed:

1. **Architecture**:
    a. **Web Platform:** Tauke.AI is a web-based platform.
    b. **Frontend:** The frontend is built using **React, Vite, JSX, and vanilla CSS**.
    c. **Backend:** The backend is built using **Python-based services**.
    d. **Database & Storage:** The system uses **Supabase**, including **PostgreSQL** for structured data and **Storage** for uploaded files.
    e. **AI Integration:** Tauke.AI integrates the **Z.AI GLM model family through an ILMU-compatible API** for documents scanning, data analysis, strategy generation, real-case scenario simulation, and executable roadmap plan generation. Mainly GLM-5.1 is used as text model while GLM-4.6v-flash is used as a vision model.
    f. **Reasoning Approach:** Instead of using one simple prompt, the backend coordinates a **Multi-Agent Debate** workflow where different AI roles evaluate the same business context from multiple perspectives.
    g. **Simulation Logic:** The system also uses **Mirofish simulation logic** to test business ideas and support more structured and realistic recommendations within the Sandbox.
    h. **External Live Context:** The system ingests real-time macro signals using OpenWeather API, Google Places API, GNews API, and BestTime API to contextualize business performance and run accurate simulations.

2. **Data Flows:**
    a. **Ingestion:** Users drag and drop sales files and supplier invoices directly into the secure cloud dashboard.
    b. **Enrichment:** The system automatically overlays external data (like local events and holidays) onto the sales numbers for full context.

c. **Extraction:** Z.ai GLM scans the messy files, extracting and organizing key financial metrics to track costs and profits.
d. **Strategy Debate:** Specialized AI personas debate different operational approaches behind the scenes to generate the safest, most profitable business strategy.
e. **Scenario Simulation:** Users can independently test their own "what-if" business ideas in a risk-free Sandbox to forecast the financial ROI.
f. **Delivery:** The winning strategy is displayed as a step-by-step roadmap with a realistic ROI forecast in a clean, interactive interface.

3. **Model Process:**
   a. **Knowledge Layer:** Building the business profile by extracting data from uploaded receipts and sales files, then layering on outside context like local public holidays to get the full picture.
   b. **Diagnostic Layer:** Scanning the historical data to automatically identify "silent killers," such as creeping ingredient costs or sudden profit drops.
   c. **Verification Layer:** Using a Supervisor Agent to pause and ask the business owner for quick confirmation if a receipt is blurry or data is missing, ensuring perfect accuracy.
   d. **Strategy & Simulation Layer:** Giving the user two distinct tools: an AI Debate where specialized personas argue behind the scenes to generate the best business strategy, and a Scenario Sandbox where users can independently test their own "what-if" ideas to forecast financial ROI.
4. **Role of Reference**:
   a. This document serves as the main technical blueprint for our developers, testers, and the hackathon judges. It provides a clear guide on how the frontend dashboard, the Supabase database, and the Z.ai GLM microservices all connect to bring the high-level architecture of Tauke.AI to life.

1.2. **Background**:

The hardest part of running a restaurant today isn't just managing the messy paperwork—it's the terrifying cost of guessing the future. When supply costs rise, owners are paralyzed by 'What if?' questions: What if I cut staff hours? What if I change my menu? Because they have no way to forecast customer reactions, they usually do nothing until it's too late, or they make a blind change that costs them thousands. Tauke.AI exists to stop owners from operating in the dark. We turn their dead data into a predictive sandbox, giving them a safe place to simulate future changes and see exactly how their profits will react before they make a single move.

1. **Previous Version:**
   a. Today, many modern F&B owners use basic tools like digital cash registers or spreadsheets. However, these tools are disconnected—your sales records are in one place, while your bills are in another. Because of this, owners can only see what went wrong in the past instead of planning for the future.
2. **Changes in Major Architectural Components**

a. We are moving away from systems that just act like a digital notebook for your numbers. Instead of only storing data, our system uses a group of AI experts to look at your business decisions from three different angles at the same time: growth, risks, and daily operations. This turns the system from a simple storage box into a tool that actually thinks and helps you plan your next move.

3. **New capabilities that are only introduced in this**
   a. **Smart File Reading:** Z.ai extracts data from messy bills, receipts, and spreadsheets directly into our database.
   b. **Checking the Neighborhood:** The system overlays your sales with local events, holidays, and news to explain exactly why your revenue changed.
   c. **Checking in with the Boss:** A "Supervisor Agent" pings you to verify data theories—like checking if a sales dip was due to a school break—ensuring 100% accuracy.
   d. **The AI Strategy Debate:** A multi-agent AI team (Profit, Risk, and Ops) debates moves behind the scenes to find the safest, most profitable strategy.
   e. **The Risk-Free Testing Area:** The Scenario Sandbox lets you "test-drive" ideas like price changes or promos with a realistic ROI forecast before spending any money..

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
F&B Merchants / Owners	They upload sales records and bills to see the "big picture" of their business health and profit	A premium, easy-to-read dashboard that shows them exactly how much they are making without needing to be a math expert.
Restaurant Operations Managers	They spot daily leaks (like rising costs) and test out promotional offers—like a "Buy 1 Free 1" deal—to see if it actually helps the business. .	Fast alerts on issues and a clear way to see if a discount or special offer will make money or just waste it.

SME Business Consultants	They use the AI-generated reports to give faster, better financial advice to multiple cafe clients at once	Automated, accurate summaries that save them hours of manual data entry and spreadsheet work.
QA Team	They look at long-term trends—like how local holidays affect sales—to find new ways for the business to grow.	Clean, organized data that is easy to export and study to find hidden opportunities for the business

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web app-based strategic decision support application for F&B SMEs
Architecture	Client-server architecture with AI service integration

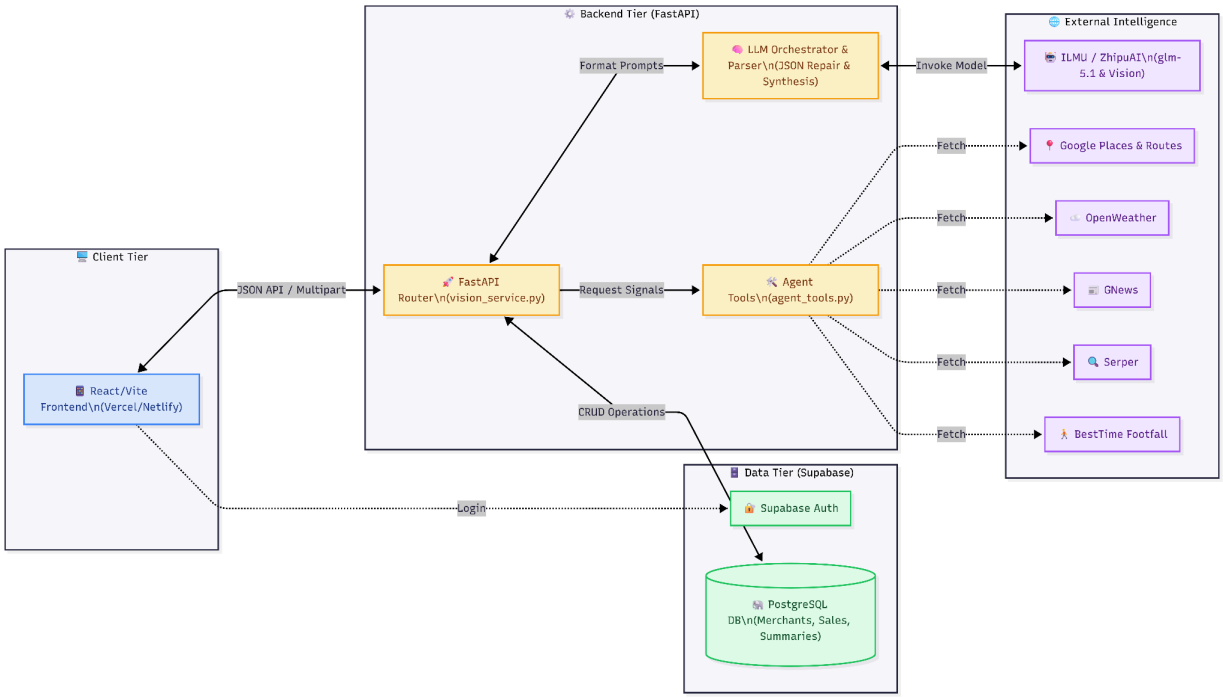
Tauke.AI is implemented as a responsive web-based system where the frontend is responsible for user interaction and guided workflow pages, while the backend coordinates file ingestion, structured data processing, external context enrichment, AI reasoning, and result delivery. The frontend is built using React, Vite, JSX, and vanilla CSS. The backend uses Python services to handle uploads, parsing, orchestration, and model calls. Supabase is used for authentication, storage, and relational persistence. The reasoning layer is powered by the Z.AI GLM model family through an ILMU-compatible API for documents scanning, data analysis, strategy generation, real-case scenario simulation, and executable roadmap plan generation. Mainly GLM-5.1 is used as text model while GLM-4.6v-flash is used as a vision model.

2.1.2 LLM as Service Layer

The LLM in Tauke.AI is used as a dedicated service layer rather than a generic AI module. The system first prepares the business context by combining merchant profile data, parsed sales records, extracted invoice or financial data, external market context, and user clarification responses before sending focused prompts to the model. This approach allows the model to be called at different workflow stages such as detective analysis, strategy generation, debate, simulation, and final recommendation. It also reduces unnecessary token usage because raw uploads are not passed directly into every model request.

2.1.3 Dependency Diagram

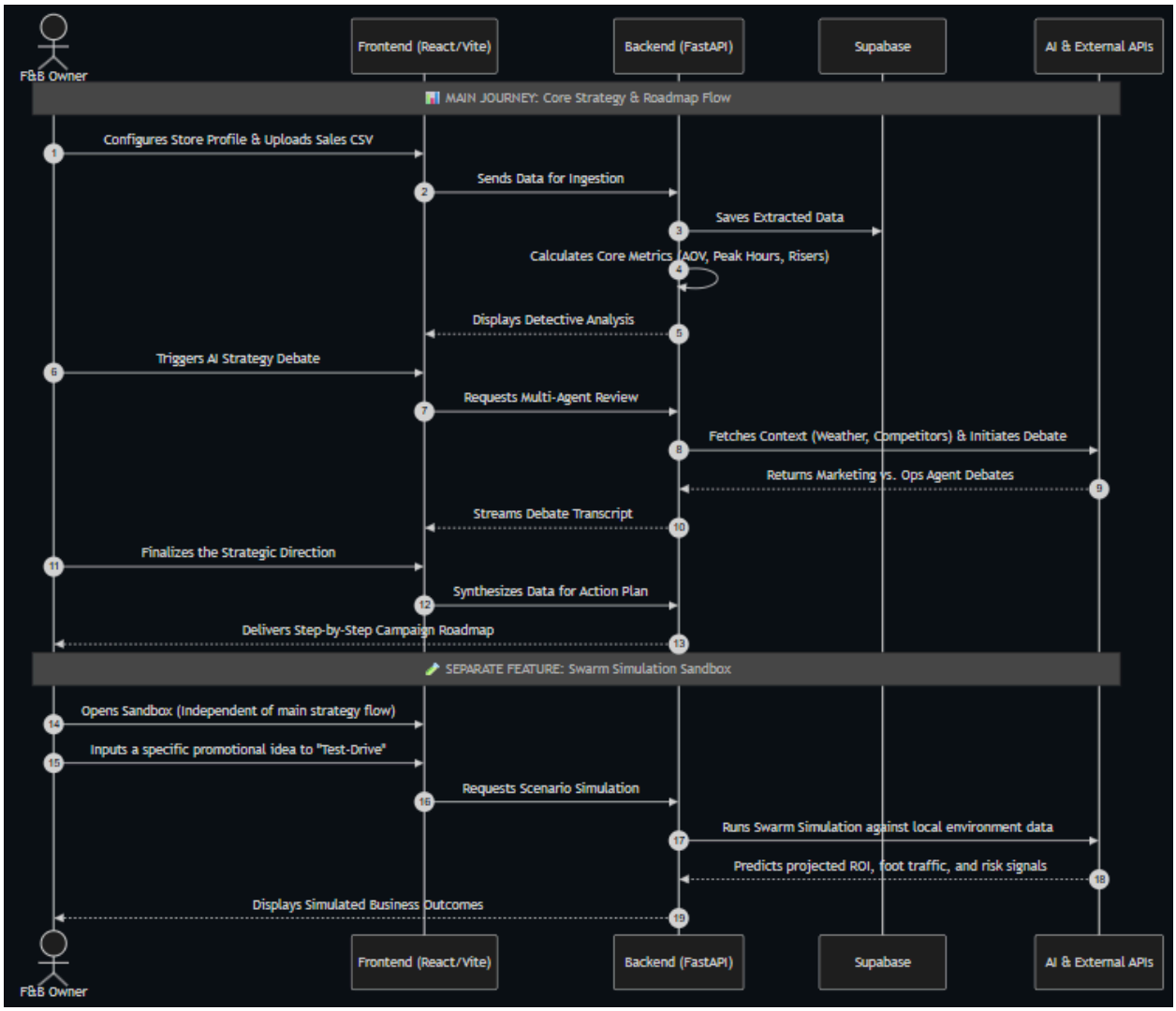
The dependency flow of Tauke.AI begins with the React frontend, where users log in, configure their business profile, upload records, review findings, and interact with strategy, debate, and simulation pages. The frontend sends requests to the Python backend, which validates inputs, stores uploaded files in Supabase Storage, parses CSV and document content, and writes structured records into Supabase PostgreSQL. The backend then collects selected external signals such as nearby business context, public holidays, events, and relevant public information. After context preparation, the backend constructs prompts and sends them to the GLM reasoning service through the ILMU-compatible API endpoint. The model response is parsed into structured outputs and routed to the next system stage or returned to the frontend. Token limits are managed before the request reaches the model by chunking or summarizing large inputs. Major API flows include frontend-to-backend requests for authentication, upload, analysis, clarification, strategy generation, simulation, and final synthesis; backend-to-Supabase requests for storage and persistence; backend-to-external-services requests for context collection; and backend-to-GLM requests for reasoning.



2.2.2. Sequence Diagram

The main interaction flow begins when the user logs in and configures the merchant profile. The user then uploads CSV sales records and PDF/image financial documents through the frontend. The backend stores the raw files, extracts usable records, and saves structured data into PostgreSQL. External context is then collected and combined with the internal business data. The backend sends this summarized context to the GLM

service for detective analysis, and the returned findings are displayed to the user. If the system needs clarification, the user responds through the Supervisor Clarification page. The backend then sends the refined context to the model for strategy generation. The user may select one generated strategy for multi-agent debate or enter one separate merchant idea into the Simulation Sandbox. Finally, the backend returns a final recommendation and roadmap, which are displayed in the Final Synthesis and Campaign Roadmap pages.



2.2. Technological Stack

2.2.1. Frontend

React + Vite + JSX + Vanilla CSS are used for the frontend. This stack supports a responsive web experience, modular page-based development, and fast local iteration. React handles UI composition, Vite supports development and build performance, JSX is used for page and component structure, and vanilla CSS is used for custom styling and workflow-specific layouts.

2.2.2. Backend

Python-based services are used for the backend to support API orchestration, business logic, upload handling, parsing, AI workflow coordination, and response delivery. The backend is responsible for validating requests, preparing structured context, integrating external services, and routing prompts to the reasoning layer.

2.2.3. Database

Supabase PostgreSQL is used as the main relational database. It stores merchant profiles, normalized business records, external context, diagnostic outputs, clarification logs, strategy options, simulation results, and final recommendations.

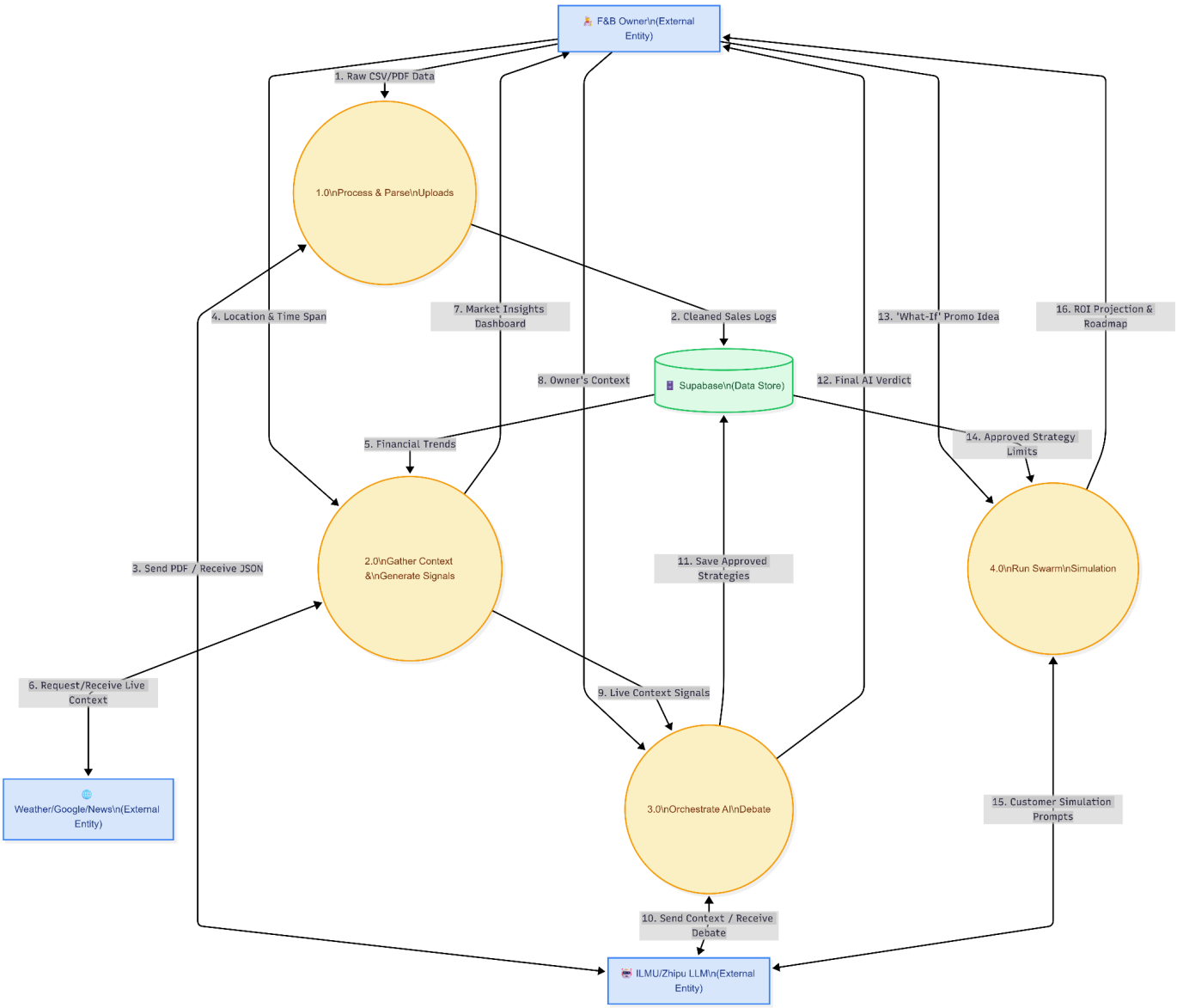
2.2.4. Cloud/Deployment

The platform utilizes a modern, decoupled cloud architecture. The frontend is deployed on **Vercel** for optimized global delivery, while the backend API services are hosted on **Render**. At the data layer, **Supabase** acts as the core infrastructure, managing user access via Supabase Auth. Structured records and AI-generated outputs are maintained in its PostgreSQL database. Finally, the application's reasoning layer operates by accessing external model APIs through an ILMU-compatible service path.

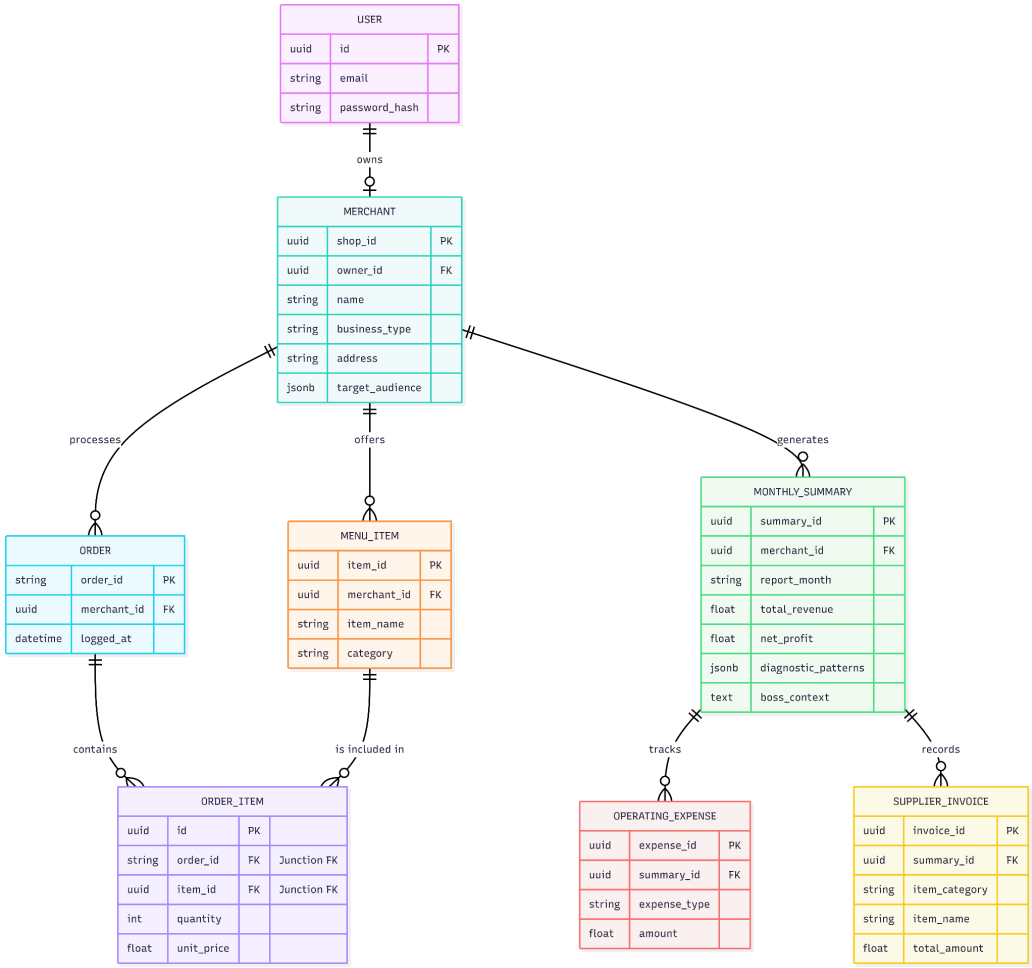
2.3. Key Data Flows

2.3.1. This part shows how data are moving through the system and how it’s structured and stored

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

#	Features	Description

1	**Business Data Upload**	Upload sales transaction records, invoices and monthly statements to automatically organize your business data.
2	**External Market Context**	Explains sales changes by checking outside factors like competitors and local events.
3	**Diagnostic Analysis Engine**	Automatically spots hidden problems like rising costs or profit leaks.
4	**Supervisor Chat Interface**	A quick chat to confirm business details so the AI's advice stays accurate.
5	**Strategy Generation**	Instantly builds multiple business plans tailored to your current situation.
6	**Multi-Agent Evaluation**	AI experts debate different strategies to pick the safest and most profitable one.
7	**Simulation Sandbox**	A risk-free "practice zone" to test your own ideas before spending any money.
8	**Final Recommendation and Roadmap**	A clear final plan with a step-by-step guide on exactly what to do next.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	Requirements	Implementation
Scalability	The system must handle high volumes of file uploads, especially during "peak hours" like month-end closing for restaurants.	Use Supabase auto-scaling for the database and Z.ai GLS elastic microservices to handle multiple AI debates at once.
Reliability	Strategy generation must be consistent. A failed AI debate should not leave the business dashboard in a broken or "half-loaded" state.	Use atomic database transactions; if an AI process fails, the system rolls back to the last "safe" save point to prevent data errors.
Maintainability	Each AI expert (Growth, Risk, Ops) must be easy to update as market conditions change in 2026.	Built using a modular Agentic Architecture, allowing developers to update a single AI persona without touching the rest of the code.
Token Latency	The **AI Strategy Debate** should provide initial feedback within 5 seconds to keep the user experience feeling "live" and fast.	Implement streaming responses (so the user sees the AI "typing" its thoughts) and use caching for repeated market context data.
Cost Efficiency	Processing a full business strategy roadmap must remain affordable for small-scale cafe owners.	Use "Prompt Engineering" to keep AI instructions concise and truncate long, repetitive sales data before sending it to the Z.ai model.

Data Security	Since the app handles sensitive financial records, all uploaded invoices must be private and secure.	Use Supabase Row Level Security (RLS) and encrypted storage buckets to ensure only the owner can see their own business data.

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026.

a. **Real-time POS integration:** A direct link between sales terminals and the analytical engine that captures every transaction instantly, transforming raw data into actionable insights without the need for manual end-of-day reporting.

b. **Automatic business execution such as changing prices or launching campaigns without user approval:** Empowering the AI to function as an autonomous operator that reacts to market shifts in real-time, executing high-impact decisions like dynamic pricing or flash sales instantly to maximize profit margins during peak or off-peak hours.

c. **Full inventory and supplier-ordering automation:** A closed-loop system that tracks stock levels in real-time and autonomously generates purchase orders based on depletion rates, ensuring the supply chain remains fluid and optimized without any human intervention.

d. **Multi-user collaborative workspace:** A shared digital environment designed for team-wide transparency, allowing owners, managers, and stakeholders to co-author business strategies, monitor live performance metrics, and streamline communication in a single, unified interface.

e.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: The MVP is first validated through internal team testing using fixed sample business records and controlled workflow checks. A limited pilot can then be run with a small number of F&B users before broader release.

Recommendation wording, presentation style, or roadmap formatting can be compared through lightweight A/B testing if time permits.

4.1.2. **Emergency Rollback (ER) & Golden Release**: If a new build causes core flow failure such as broken uploads, invalid AI outputs, or unavailable backend services, the team should revert to the last stable version that passed internal testing. Advanced AI-driven features can be temporarily disabled while keeping basic upload and review flows available. Logs from the failed build should be reviewed before redeployment.

4.1.3. Priority Matrix:

4.1.3.1. Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project.

Priority	Trigger Conditions	Actions
P1 Critical	Login, upload, or backend service unavailable for core demo flow	Roll back to the last stable build and disable non-essential features
P2 High	LLM output invalid, incomplete, or inconsistent with input data	Retry with safer prompt, validate response, and show fallback message if needed
P3 Medium	Analysis latency becomes too slow during heavy prompts or repeated testing	Reduce prompt size, reuse cached context, and simplify multi-agent depth
P4 Low	UI inconsistency or minor visual issue without affecting system logic	Record issue and fix in the next patch

4.2. Monitoring:

4.2.1. Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.

The team will monitor upload success rate, parsing completion, model error rate, response latency, and Supabase storage/database health. Repeated upload failure will trigger an ingestion check. Empty or malformed extraction results will trigger parsing review. Repeated model/API errors will trigger fallback handling and prompt review. Excessive response time will trigger prompt-size reduction or staged processing. Storage or retrieval failures will trigger backend and Supabase connectivity checks.

4.3. Assumptions:

a. Users have stable internet access during upload and analysis
b. Uploaded CSV, PDF, or image files are readable and relevant to the business
c. External context sources are reachable during analysis
d. Final business decisions will still be reviewed by the human user

4.4. External Dependencies:

Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z.ai GLM / ILMU-compatible API	Core reasoning engine for detective analysis, strategy generation, debate, simulation, and final synthesis	High risk if the API is unavailable or rate-limited; mitigation is graceful fallback, retry handling, and staged processing
Supabase	Database, storage, and authentication	High risk if connectivity fails; mitigation is validation checks and preservation of critical demo data
Google Places API	Competitor and nearby business context	Medium risk if unavailable; mitigation is continue with internal-only analysis
OpenWeather API	Provides weather context that may explain changes in customer traffic or sales patterns.	Medium risk if unavailable or location lookup fails; mitigation is to continue analysis without weather signals.
GNews API	Provides news, local event, and macro-context signals that may affect business performance.	Medium risk if the API is unavailable or returns limited results; mitigation is to cache recent results and continue with partial external context.

Serper API	Provides Google Search-based web context for competitor promotions, unofficial local events, and public market signals.	Medium risk if rate-limited or unavailable; mitigation is to use other available context sources or continue with internal-only analysis.
Google Routes / BestTime API	Provides traffic, route, or footfall-related signals to support customer movement and demand estimation.	Medium risk if unavailable or quota-limited; mitigation is to use default traffic assumptions and continue with sales-based analysis.
Vercel	Hosts and serves the React + Vite frontend web application.	High risk if the deployment is unavailable or frontend build fails; mitigation is to redeploy the latest stable build and keep the GitHub repository updated
Render	Hosts the FastAPI backend service used by the deployed frontend.	High risk if the backend sleeps, fails to start, or environment variables are missing; mitigation is to wake the health endpoint before demo, check logs, and redeploy the latest stable backend.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day 1 focuses on problem framing, domain selection, and feature scoping. Day 2 focuses on user flow planning, wireframing, and architecture drafting. Day 3 focuses on database design, Supabase setup, and backend structure. Day 4 focuses on CSV, invoice, and financial document ingestion logic. Day 5 focuses on external context collection and detective analysis flow. Day 6 focuses on supervisor clarification, strategy generation, and debate logic. Day 7 focuses on simulation sandbox, final synthesis, and roadmap generation. Day 8 focuses on frontend integration and responsive UI refinement. Day 9 focuses on testing, debugging, and documentation. Day 10 focuses on pitch preparation, demo rehearsal, and final submission.

5.2. Team Members Role:

Lim Jun Bin – Core project Manager / AI / Reasoning Lead – prompt flow, structured output handling, debate logic, simulation logic, and synthesis

Vincent Loh Yong Sheng – Frontend / UI-UX Lead – React + Vite workflow pages, responsive UI, navigation, and interface quality
Goh Pei Jia – Backend / API Lead – Python backend services, orchestration logic, integration endpoints, and core request handling
Daniel Goh Zhi Qian – Data / Database / QA Lead – Supabase schema, storage structure, test data, validation support, and documentation
Yu Yong Jiang - Research & Presentation Lead – Responsible for hackathon market research, verifying business logic, ensuring alignment with competition criteria, and designing the final pitch deck (slides) for the presentation.

5.3. Recommendations:

- Use reusable prompt templates to keep output structure consistent across stages
- Cache stable external context where possible to reduce repeated token and API usage
- Validate extracted records before sending them to the reasoning layer
- Keep strategy generation and simulation as separate flows to reduce user confusion
- For future scaling, move heavy extraction and multi-step reasoning into background job processing